

FacePass Android version V3.8.0 application strategy description

-
- [1. Access control/attendance business process](#)
 - [1.1 Base library management](#)
 - [1.2 Face detection, recognition/live body](#)
- [2. Human ID machine 1:1 business process](#)
- [3. Application of attribute model in business](#)
 - [3.1 Introduction to the attribute model](#)
 - [3.2 Introduction to Property Model Pipeline](#)
 - [3.3 Usage of attribute model](#)
 - [3.3.1 Configuration and detection related data structure](#)
 - [3.3.2 Identification or living body related data structure](#)
 - [3.3.3 Simple application](#)
- [4. Multi-base library recognition](#)
 - [4.1 Multi-base library identification process](#)
 - [4.2 Multi-base library recognition result analysis](#)
- [5. The usage of long distance mode](#)
- [6. mcvSafe Authorization](#)

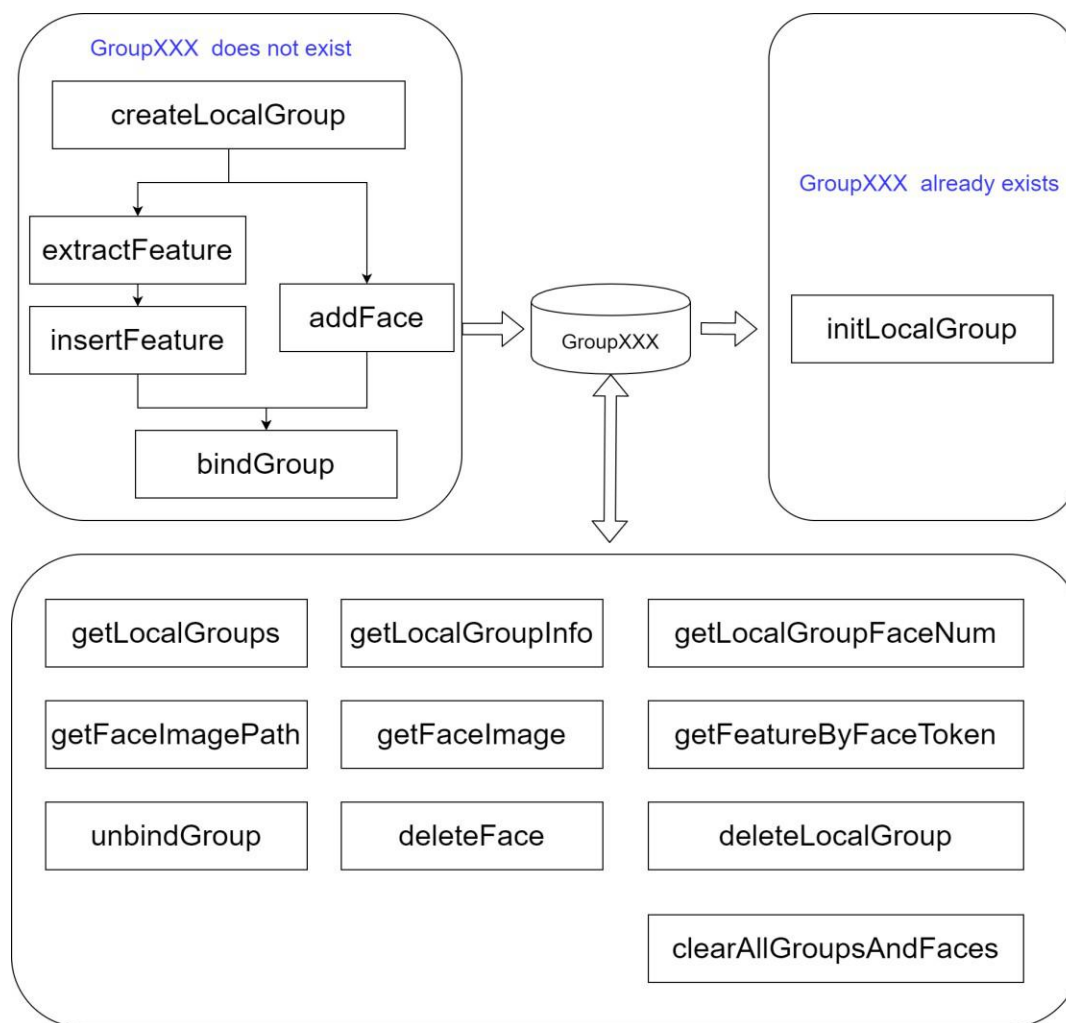
1. Access control/attendance business process

Access control/attendance is usually composed of three key links: detection, identification, and living body, among which identification is a 1:N comparison, and living body can be used with monocular and binocular modules. In the case of monocular, the RGB live model is used, while for binoculars, the RGBIR live model is used. Other models are the same.

The usual business process is mainly divided into: base database management and face detection, recognition, and living body.

1.1 Base library management

Bottom library management In monocular and binocular applications, the interfaces are basically the same. There are two sets of interfaces for users to choose from in the bottom library, namely addFace and extractFeature+insertFeature, as follows:



If the base library does not exist, you need to call the library entry interface to add the face to the base library. If the base library already exists, you can directly call the initLocalGroup interface to complete the initialization and load the face information in the base library into the memory.

In the two sets of warehousing interfaces, the addFace interface has its own face cutout function and stores the face cutout in the sdcard. Users can obtain the face cutout information through interfaces such as getFacelImage*. The combination of extractFeature+insertFeature does not have a face cutout function. Users can cut out the face frame information returned by extractFeature, and then pass the cutout information to the insertFeature interface. It is an optional feature to store face cutouts in the base library. If users manage face cutouts by themselves, they do not need to pass in the face cutout information to insertFeature, which can improve the efficiency of warehousing. Users can also combine the SDK on the x86 platform to extract features on x86 and deliver the features to each terminal. The terminal calls the insertFeature+bindGroup interface to implement storage and binding, which further improves storage efficiency and unified management of multi-terminal base libraries .

Please refer to the developer's guide for detailed usage of related interfaces of storage.

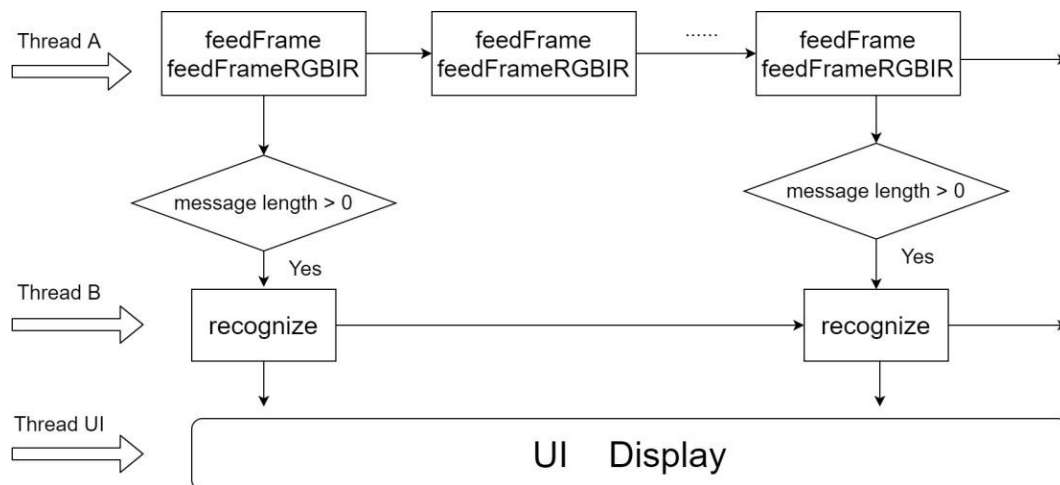
1.2 Face detection, recognition/live body

For ease of use, the recognition and living of the SDK are implemented in a recognize interface.

When configuring the FacePassConfig switch for monocular/binocular live detection, there are the following differences:

- Monocular: config.livenessEnabled = true; config.rgbIrlivenessEnabled = false;
- Binocular: config.livenessEnabled = false; config.rgbIrlivenessEnabled = true;

The recommended process is as follows:



Face detection and recognition/live body, it is recommended to call in two independent threads in parallel to improve efficiency. The threads communicate through a queue, and the queue stores the detection results. For details, please refer to the simple implementation in the demo.

The result returned by face detection consists of three parts: message (encrypted information of the face through the quality filter, input to the recognition interface), faceList (information of all faces detected in the current data frame, including the faces that pass and fail the quality), images (pass the quality filter, wait Send recognized face matting information). Images save the matting information of the face to be sent, so it corresponds to the message. If the message is empty, the images must also be empty. The message is not empty. The images recorded in the images are the cutout information to be sent to recognize the face, but the cutout is not used for recognition, and the original image cached in the SDK is used for recognition.

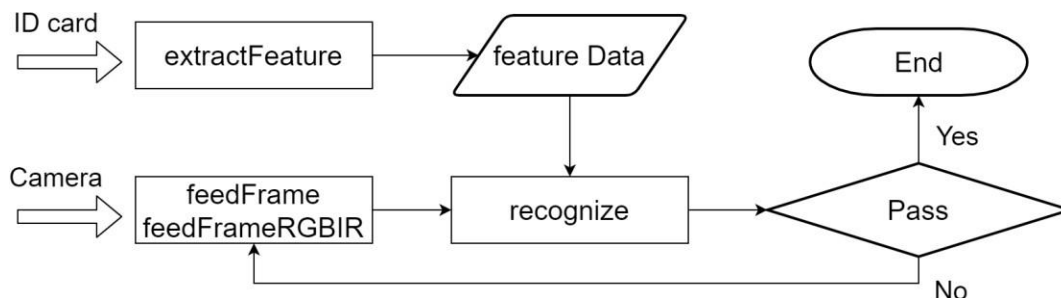
The results returned by face recognition mainly have three states: RECOGNITION_PASS (recognition passed), RECOGNITION_RETRY_EXPIRED (recognition failure without retry times), RECOGNITION_RETRY (recognition failure with retry times). For more details, please refer to the developer guide.

When the face of a trackid is in the process of recognition, the face detection of the trackid will not generate a new message. During the recognition period, the trackid will always be in the state of pushing the picture. When the recognition is completed and the recognition result is a failure, a retry is required, The face detection of the trackid will generate a new message, which is the best push image during the last recognition.

When a trackid has passed the recognition (status RECOGNITION_PASS), or the recognition fails without retry times (RECOGNITION_RETRY_EXPIRED), the face detection of the trackid will not generate a new message. If the user needs to start the recognition of the trackid's face again, You can call the interface resetMessage interface (Note: V3.9.0 and later will be changed to setMessage interface).

2. Human ID machine 1:1 business process

The process of the identification machine does not need the SDK's base library management function. The recommended business process is as follows:



The recognize interface used by the human identification machine business process does not pass in the base library + message, but the facial feature data + message (referred to as the recognition interface of the baseless library). For the specific interface and return value description, please refer to the developer guide.

3. Application of attribute model in business

3.1 Introduction to the attribute model

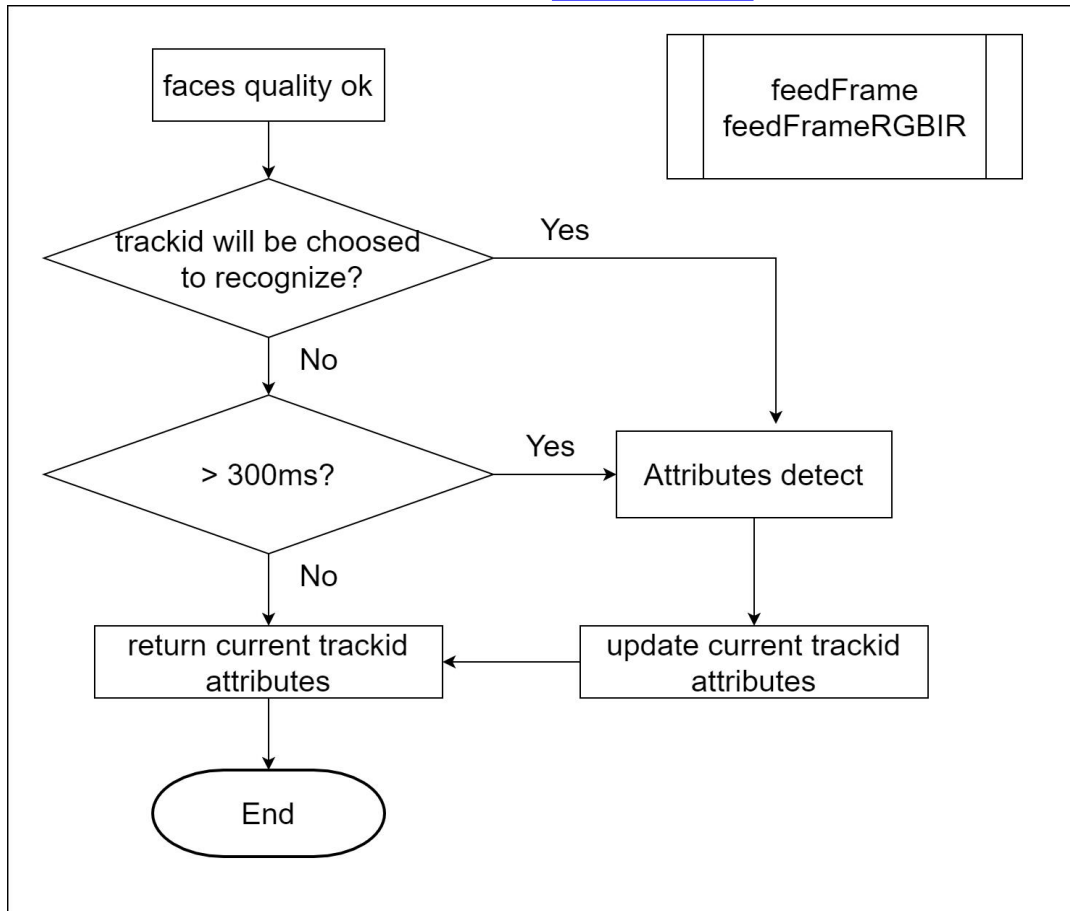
The attribute model outputs 6 types of attribute multi-category, namely:

- Hair: Bald head, small amount of hair (including baldness), short hair, long hair, unknown
- Beard: No beard or not obvious beard, beard on the lips, beard, unknown
- Hats: no hats, safety helmets, chef hats, student hats, helmets, all ethnic security 01 Little white hat, turban, other hats, unknown
- Masks: no masks, medical masks, haze masks, ordinary masks, transparent masks, unknown
- Glasses: no glasses, dark-rimmed transparent glasses, ordinary transparent glasses, sunglasses, unknown
- Skin tone: yellow skin, white skin, dark skin, dark skin, unknown

Please refer to FacePassRCAttribute class. In the FacePassRCAttribute class, each attribute adds an INVALID field to indicate that the value is invalid. It may be that the attribute model is not enabled or the attribute detection is abnormal.

3.2 Introduction to Property Model Pipeline

Attribute model detection is included in the feedFrame/feedFrameRGBIR interface. The strategy is to perform attribute detection for faces that have passed quality filtering at intervals greater than or equal to 300ms. During 300ms, historical attribute information will be used directly. For the trackid sent for recognition, attributes will be mandatory. Detection. The process is as follows:



3.3 Usage of attribute model

3.3.1 Configuration and detection related data structure

```

FacePassConfig.java
public FacePassModel rcAttributeModel; public boolean
rcAttributeEnabled;
FacePassRCAttribute.java
public class FacePassRCAttribute {
  public FacePassHairType hairType;
  public FacePassBeardType beardType;
  public FacePassHatType hatType;
  public FacePassRespiratorType respiratorType;
  public FacePassGlassesType glassesType;
  public FacePassSkinColorType skinColorType;
}
  
```

```

...
}
FacePassFace.java
public class FacePassFace {
    public long trackId;
    public FacePassRect rect;
    public FacePassPose pose;
    public float blur;
    public FacePassRCAttribute rcAttr; //Only faces that pass the quality filter contain valid attribute
    information
    public float smile;
    public int facePassFeedFrameErrorCode; //Whether the face is of high quality

    ...
}
FacePassImage.java
public class FacePassImage {
    public byte[] image;
    public int width;
    public int height;
    public int facePassImageRotation;
    public int facePassImageType;
    public long trackId;
    public FacePassRCAttribute rcAttr; //Send the attribute information of the recognized face
    ...
}

```

The FacePassFace[] faceList and FacePassImage[] images of the return value of the feedFrame/feedFrameRGBIR interface both contain the attribute information of the face. The difference is:

- The faceList will contain all faces detected in the current frame, but among these faces, only the attribute values of the faces that pass the quality filter are valid; that is, if the facePassFeedFrameErrorCode of the trackid is not 0, then rcAttr is invalid.
- The face attribute in images is the attribute to be sent to recognize the face. If the user wants to change the recognition threshold of the face based on the face attribute, he can judge by the face attribute here. We will introduce the usage later.

3.3.2 Identification or living body related data structure

FacePassTrackOptions.java

```
public class FacePassTrackOptions {
    public long trackId;
    public float searchThreshold; //0.0~100.0, other values are invalid
    public float livenessThreshold; //0.0~100.0, other values are invalid
    ...
}
FacePassRecognitionDetail.java
public class FacePassRecognitionDetail {
    public float searchScore;
    public float searchThreshold;
    public float livenessScore;
    public float livenessThreshold;
    public FacePassRCAttribute rcAttr; //The attribute information here is taken directly from the test results
    ...
}
FacePassLivenessResult.java
public class FacePassLivenessResult {
    public long trackId;
    public float livenessScore;
    public float livenessThreshold;
    public int livenessState;
    public int livenessErrorCode;
    public FacePassRCAttribute rcAttr; //The attribute information here is taken directly from the test results
    ...
}
```

The recognize/livenessClassify has an overloaded interface that supports changing the current recognition/liveness threshold. For details, please refer to the developer guide.

3.3.3 Simple application

In the following scenarios with and without a mask, in order to increase the recall rate, different recognition thresholds are used to illustrate a simple application of the attribute model. The specific requirements are as follows:

- Do not wear a mask, use the recognition threshold 72.0
- Wear a mask and use the

recognition threshold 69 1) Initialize the

handle and enable the attribute model


```
config.rcAttributeModel =  
FacePassModel.initModel(getApplicationContext().getAssets(),  
"attr.RC.gray.12M.arm.200229.bin"); config.rcAttributeEnabled = true;  
config.searchThreshold = 72f; // The default threshold for not wearing a mask  
...  
...  
mFacePassHandler = new FacePassHandler(config);
```

2) Build a queue for data communication between feedFrame/feedFrameRGBIR thread and recognize thread

```
public class DetectMessage{  
    public byte[] message; //Send the recognized message  
    public FacePassTrackOptions[] trackOpt; //Threshold  
  
    public DetectMessage(byte[] message) {  
        this.message = message;  
        this.trackOpt = null;  
    }  
  
    public DetectMessage(byte[] message, FacePassTrackOptions[] opt) {  
        this.message = message;  
        this.trackOpt = opt;  
    }  
}  
  
ArrayBlockingQueue<DetectMessage> detectMessageaQueue = new  
ArrayBlockingQueue<DetectMessage>(5);  
  
...
```

3) feedFrame/feedFrameRGBIR thread

This thread continuously reads the camera data frame and performs face detection. When the face meets the recognition conditions, it will return a valid message. We can construct a DetectMessage for the face based on the face attribute information and put it in the detectMessageaQueue In the queue.

```
//detectionResult = mFacePassHandler.feedFrame(image); detectionResult =
mFacePassHandler.feedFrameRGBIR(imageRGB, imageIR); if (detectionResult != null &&
detectionResult.message.length != 0) {
    FacePassTrackOptions[] trackOpts = new FacePassTrackOptions[detectionResult.images.length];
    // Through detectionResult.images.length, we can know the face information of several trackids in
    the message
    for (int i = 0; i < detectionResult.images.length; ++i) {
        if (detectionResult.images[i].rcAttr.respiratorType !=
        FacePassRCAttribute.FacePassRespiratorType.NO_RESPIRATOR
        && detectionResult.images[i].rcAttr.respiratorType !=
        FacePassRCAttribute.FacePassRespiratorType.UNKNOWN
        && detectionResult.images[i].rcAttr.respiratorType !=
        FacePassRCAttribute.FacePassRespiratorType.INVALID) {
            float searchThreshold = 69f; // For faces with masks, lower the recognition threshold
            float livenessThreshold = -1.0f; // -1.0f will not change default liveness threshold
            trackOpts[i] = new FacePassTrackOptions(detectionResult.images[i].trackId, searchThreshold,
            livenessThreshold);
        }
        /*Print attribute information*/
        Log.d(DEBUG_TAG, String.format("rc attribute hairType: 0x%x beardType: 0x%x hatType: 0x%x
        respiratorType: 0x%x glassesType: 0x%x skinColorType: 0x%x",
        detectionResult.images[i].rcAttr.hairType.ordinal(),
        detectionResult.images[i].rcAttr.beardType.ordinal(),
        detectionResult.images[i].rcAttr.hatType.ordinal(),
        detectionResult.images[i].rcAttr.respiratorType.ordinal(),
        detectionResult.images[i].rcAttr.glassesType.ordinal(),
        detectionResult.images[i].rcAttr.skinColorType.ordinal()));
    } // end of loop

    /*Put into the queue*/
    DetectMessage mDetectMessage = new DetectMessage(detectionResult.message, trackOpts);
    detectMessageQueue.offer(mDetectMessage);
}
```

4) recognize thread

```
DetectMessage mDetectMessage = detectMessageQueue.take();

FacePassRecognitionResult[] recognizeResult = mFacePassHandler.recognize(group_name,
mDetectMessage.message, mDetectMessage.trackOpt);
if (recognizeResult != null && recognizeResult.length> 0) {
    for (FacePassRecognitionResult result: recognizeResult) {
        if(result.facePassRecognitionState == FacePassRecognitionState.RECOGNITION_PASS) {
            ...
        }
        if(result.facePassRecognitionState == FacePassRecognitionState.RECOGNITION_RETRY_EXPIRED) {
            ...
        }
    }

    /*Print the recognition result. The attribute information here is the same as the attribute information
    returned by the feedFrame/feedFrameRGBIR interface. By printing the result.detail.searchThreshold
    value, we can confirm whether the threshold value we changed is effective*/
    Log.d(DEBUG_TAG, String.format("recognize trackid: %d,
searchScore: %f searchThreshold: %f, hairType: 0x%x beardType: 0x%x hatType: 0x%x respiratorType:
0x%x glassesType: 0x%x skinColorType: 0x%x",
result.trackId,
result.detail.searchScore,
result.detail.searchThreshold,
result.detail.rcAttr.hairType.ordinal(),
result.detail.rcAttr.beardType.ordinal(),
result.detail.rcAttr.hatType.ordinal(),
result.detail.rcAttr.respiratorType.ordinal(),
result.detail.rcAttr.glassesType.ordinal(),
result.detail.rcAttr.skinColorType.ordinal()));
}
}
```

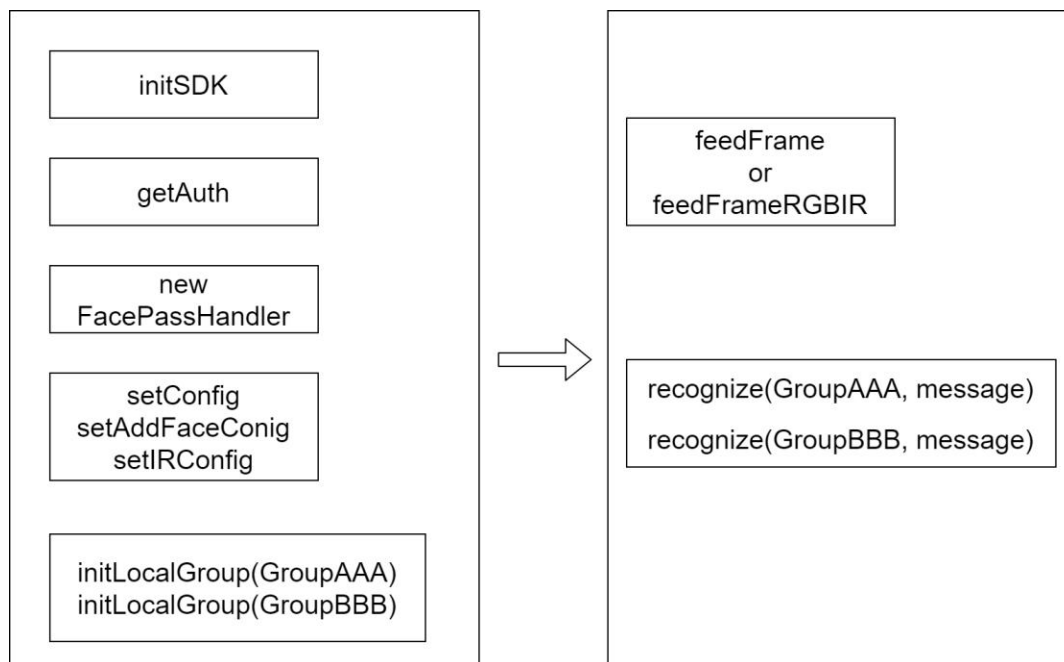
4. Multi-base library recognition

The sdk supports the same message to be recognized between multiple base libraries, but because the recognition results between multiple base libraries will affect the state of the face, please make sure that you know the following details, and Differences in single-bottom library recognition.

The V3.9.0 version will optimize this function and increase the speed. If you use the multi-base library recognition function now, please continue to pay attention to the changes in subsequent versions of this function.

4.1 Multi-base library recognition process

The process of multi-base library identification is to pre-load multiple base libraries into the memory through initLoaclGroup. When the identification interface is called, different base library names are passed in to obtain the recognition results under different base libraries, and then based on different base libraries. The recognition result of the library is processed for different business.



4.2 Multi-base library recognition result analysis

1) First understand the process of single-bottom library recognition:

The status returned by the trackid identification is RECOGNITION_PASS, then the trackid will not generate new messages in the subsequent feedFrame/feedFrameRGBIR (unless the resetMessage interface is adjusted to reset the trackid status);

The status returned by the trackid identification is RECOGNITION_RETRY_EXPIRED, then the trackid will not generate a new message in the subsequent feedFrame/feedFrameRGBIR (unless the resetMessage interface is adjusted to reset the trackid status);

The status returned by the trackid identification is RECOGNITION_RETRY, indicating that there are more attempts, then the trackid will generate a new message in the subsequent feedFrame/feedFrameRGBIR;

2) Under multi-base library recognition

The recognition result of each base library will change the current trackid status. As long as the recognition result of one base library is passed, the status of the trackid will not be updated inside the SDK. Therefore, the result of multiple base library recognition cannot be simply relied on. The state of facePassRecognitionState is judged. For example, if we first call recognize(groupAAA, message) and the result facePassRecognitionState is equal to RECOGNITION_PASS, if we call recognize(groupBBB, message) again, regardless of the recognition score, facePassRecognitionState is still equal to RECOGNITION_PASS, so we must judge with searchErrorCode. Only when facePassRecognitionState is equal to RECOGNITION_PASS and searchErrorCode is equal to SEARCH_OK_CODE, can it be confirmed that the trackid has been recognized in this base library.

Calling the same message in multiple base libraries will not consume retry times repeatedly. Please be aware of the following details:

	The same message calls recognize multiple times	Description
Situation 1: retry=5, the face information contained in the message is not in groupAAA and groupBBB	recognize(groupAAA, message) the recognition result RECOGNITION_RETRY, call recognize(groupBBB, message) again, the recognition result is still RECOGNITION_RETRY	Two calls After recognize, the number of retry remaining 4, the trackid will generate a new message

Situation 2: retry=5, the face information contained in the message is in groupAAAA, but not in groupBBB	recognize(groupAAA, message) recognition result RECOGNITION_PASS, after calling recognize(groupBBB, message) again, because the face trackid corresponding to the message has been set to RECOGNITION_PASS in the first recognition of groupAAA, the status returned by the second recognition is still RECOGNITION_PASS, but the second recognition score will definitely be lower than the threshold, searchErrorCode will indicate that the recognition score is low	The retry is still 5. After the recognition is passed, the retry will not change. TheAfter the recognition status of trackid is set to RECOGNITION_PASS, The subsequent recognition is still RECOGNITION_PASS status, and To determine the true situation of subsequent identification, you must combine the error code searchErrorCode. TheTrackid recognition has passed, no new message will be generated
Situation 3: retry=5, the face information contained in the message is in groupBBB, but not in groupAAA	recognize(groupAAA, message) The recognition result RECOGNITION_RETRY, after calling recognize(groupBBB, message) again, the recognition result RECOGNITION_PASS	retry remaining 4 TheTrackid recognition has passed, no new message will be generated

5. Usage of long distance mode

FacePassConfig added detectMode parameter in v3.8.0 version, 0—normal detection mode, 1—Long-distance detection mode (small face detection mode).

The default value is 0, and other illegal values passed in are treated as default values. When the remote detection mode is turned on, the detection model can detect smaller faces in the picture, but at the same time it will sacrifice speed.

The affected interfaces are: feedFrame, feedFrameRGBIR, addFaceDetect, these three interfaces can use setConfig to control the detectMode parameter; detectFaces, addFace, these two interfaces can control the detectMode parameter through setAddFaceConfig.

6. mcvSafe Authorization

Pls reference "FacePass for Android McvSafe Soft Authorization Operation Guide V3.8.0"